

Referat

Proiect telemedicină

Alupoaie Darius

17.05.2026

Pe scurt

Aplicația este făcută în JavaFX și Oracle și simulează un sistem de telemedicină. Pacientul își poate verifica abonamentul, completează o evaluare cu simptome, iar baza de date generează un diagnostic provizoriu. Pentru cazurile simple se poate genera automat o fișă medicală, iar pentru cazurile mai complexe se caută automat o programare la un doctor.

Cuprins

1	Descrierea proiectului	2
2	Cerințe de business identificate	2
3	Normalizarea bazei de date	3
4	Diagrama E/A sub formă UML	6
5	Entități, asocieri și chei	6
6	Constrângeri folosite	7
7	Scriptul de creare a tabelor	8
8	Logică PL/SQL implementată	9
9	Prezentarea aplicației	11
10	Concluzie	13

1 Descrierea proiectului

Proiectul ales este un sistem de telemedicină, adică o aplicație prin care pacientul poate primi o primă evaluare medicală fără să fie neapărat prezent fizic la cabinet. Aplicația are două tipuri de utilizatori: pacient și doctor. Pacientul pornește evaluarea de simptome, iar doctorul vede programările și completează fișa medicală finală.

Baza de date nu este folosită doar pentru stocare. O parte din logica aplicației este pusă în PL/SQL: verificarea abonamentului, validarea datelor, generarea diagnosticului provizoriu și căutarea unui interval liber pentru programare. Din motivul acesta, proiectul este potrivit pentru o demonstrație de normalizare, deoarece apar multe entități care trebuie separate corect: pacienți, abonamente, simptome, reguli, evaluări, diagnostice, doctori, programări și fișe medicale.

2 Cerințe de business identificate

Cerințele de mai jos sunt cele după care am gândit aplicația și baza de date. Unele vin direct din tema proiectului, iar altele apar din felul în care am ales eu să implementez sistemul de telemedicină.

Pacienți și acces în platformă

- un pacient are date de identificare, date de contact și data nașterii;
- CNP-ul, telefonul și emailul trebuie să fie unice, ca să nu apară conturi duplicate pentru aceeași persoană;
- pentru pacienții minori se rețin și datele tutorelui;
- pacientul poate avea abonament activ, expirat sau anulat;
- evaluarea medicală poate fi pornită doar dacă există un abonament activ;
- pentru pacient se pot reține afecțiuni cunoscute, de exemplu diabet, hipertensiune, alergii sau alte probleme cronice.

Evaluarea medicală automată

- pacientul completează inițial un simptom principal și două simptome secundare opționale;
- simptomele sunt luate dintr-un catalog, ca să nu fie scrise diferit de la o evaluare la alta;
- o regulă de diagnostic poate fi asociată cu mai multe simptome;
- sistemul alege o regulă potrivită și generează un diagnostic provizoriu;
- complexitatea cazului poate crește dacă pacientul este copil, vârstnic sau are afecțiuni cronice;

- pentru o evaluare se salvează diagnosticul automat obținut.

Programări și consultații

- pentru cazurile simple se poate crea automat o fișă medicală cu recomandări de bază;
- pentru cazurile mai complexe se caută automat un doctor cu specializarea potrivită;
- doctorii au program de lucru, cu oră de început și oră de sfârșit;
- o programare leagă un pacient de un doctor la o anumită dată și oră;
- programarea nu poate fi creată în trecut și trebuie să aibă durată minimă;
- după consultație, doctorul poate completa diagnosticul final, rețeta, recomandările și trimiterea către spital.

3 Normalizarea bazei de date

Pentru partea de normalizare am pornit de la o relație mare, ca în exercițiile făcute la curs. În relația aceasta am pus, teoretic, toate informațiile pe care aplicația le-ar putea ține într-un singur tabel. Evident, nu ar fi o variantă bună pentru implementare, dar ajută la demonstrație.

Pentru a nu scrie toate denumirile coloanelor de fiecare dată, am folosit următoarele notații:

Simbol	Semnificație	Simbol	Semnificație
P	pacient	DP	date pacient
AB	abonament	DAB	date abonament
AF	afecțiune pacient	DAF	date afecțiune
S	simptom din catalog	DS	date simptom
RG	regulă diagnostic	DRG	date regulă
E	evaluare simptome	DE	date evaluare
DG	diagnostic automat	DDG	date diagnostic
DOC	doctor	$DDOC$	date doctor
PR	programare	DPR	date programare
FM	fișă medicală	DFM	date fișă

Relația inițială este:

$$R = \{P, DP, AB, DAB, AF, DAF, S, DS, RG, DRG, E, DE, DG, DDG, DOC, DDOC, PR, DPR, FM, DF, DFM\}$$

Pentru evaluare am notat separat simptomele alese în formular ca S_1 , S_2 și S_3 . Ele nu sunt o listă nelimitată, ci trei poziții fixe: simptom principal, simptom secundar 1 și simptom secundar 2.

Dependențele funcționale pe care le-am folosit sunt:

$$\begin{aligned}
P &\rightarrow DP \\
AB &\rightarrow P, DAB \\
AF &\rightarrow P, DAF \\
S &\rightarrow DS \\
RG &\rightarrow DRG \\
E &\rightarrow P, S_1, S_2, S_3, DE \\
DG &\rightarrow E, RG, DDG \\
DOC &\rightarrow DDOC \\
PR &\rightarrow P, DOC, DPR \\
FM &\rightarrow PR, DFM
\end{aligned}$$

Dependențele multivaluate care apar în mod natural în proiect sunt:

$$P \twoheadrightarrow AB, \quad P \twoheadrightarrow AF, \quad RG \twoheadrightarrow S$$

Adică un pacient poate avea mai multe abonamente în timp și mai multe afecțiuni, iar o regulă de diagnostic poate fi legată de mai multe simptome. Acestea sunt dependențe multivaluate reale, fiindcă valorile respective nu sunt doar niște câmpuri simple ale pacientului sau ale regulii, ci pot apărea de mai multe ori.

Ca să verific cheia pe relația mare, se poate lua o cheie candidat de forma:

$$K = \{AB, AF, S, DG, FM\}$$

Închiderea ei acoperă relația inițială:

$$\begin{aligned}
K^+ &= \{AB, AF, S, DG, FM\} \\
&\Rightarrow P, DAB, DAF, DS, E, RG, DDG, PR, DFM \\
&\Rightarrow DP, DRG, S_1, S_2, S_3, DE, DOC, DPR \\
&\Rightarrow DDOC
\end{aligned}$$

Deci K^+ ajunge să conțină toate atributele din relația inițială. Totuși, relația R nu este în BCNF, deoarece există dependențe în care partea stângă nu este supercheie. De exemplu:

$$P \rightarrow DP, \quad S \rightarrow DS, \quad DOC \rightarrow DDOC$$

În aceste cazuri, P , S și DOC determină doar informațiile proprii, nu toată relația. De aici apar anomaliile: dacă un pacient are mai multe evaluări, datele lui s-ar repeta; dacă se schimbă telefonul doctorului, ar trebui modificat în mai multe rânduri; dacă un simptom apare în mai multe reguli, descrierea lui s-ar repeta.

Descompunerea după BCNF se poate face alegând pe rând dependențele care încalcă forma normală. Scris compact, rezultă:

$R_1(P, DP)$	din $P \rightarrow DP$
$R_2(AB, P, DAB)$	din $AB \rightarrow P, DAB$
$R_3(AF, P, DAF)$	din $AF \rightarrow P, DAF$
$R_4(S, DS)$	din $S \rightarrow DS$
$R_5(RG, DRG)$	din $RG \rightarrow DRG$
$R_6(E, P, S_1, S_2, S_3, DE)$	din $E \rightarrow P, S_1, S_2, S_3, DE$
$R_7(DG, E, RG, DDG)$	din $DG \rightarrow E, RG, DDG$
$R_8(DOC, DDOC)$	din $DOC \rightarrow DDOC$
$R_9(PR, P, DOC, DPR)$	din $PR \rightarrow P, DOC, DPR$
$R_{10}(FM, PR, DFM)$	din $FM \rightarrow PR, DFM$

După această descompunere, relațiile obținute corespund tabelelor din baza de date:

Relație	Tabela din proiect
R1	PACIENTI
R2	ABONAMENTE
R3	AFECTIUNI_PACIENT
R4	SIMPTOME_CATALOG
R5	REGULI_DIAGNOSTIC
R6	EVALUARI_SIMPTOME
R7	DIAGNOSTICE_AUTOMATE
R8	DOCTORI
R9	PROGRAMARI
R10	FISA_MEDICALA

Pentru 4NF mai trebuie tratată partea multivaluată. În relația mare, dacă aș ține în același loc pacientul, abonamentele și afecțiunile lui, ar apărea combinații inutile între fiecare abonament și fiecare afecțiune. De aceea informațiile sunt separate:

$$R(P, AB, AF) \Rightarrow R_2(AB, P, DAB) \quad \text{și} \quad R_3(AF, P, DAF)$$

La fel, o regulă poate avea mai multe simptome. Dacă aș pune simptomele direct în tabela de reguli, fie aș avea coloane de tip `id_simptom_1`, `id_simptom_2`, fie aș repeta regula de mai multe ori. Varianta normalizată este tabela de legătură:

$$R(RG, S) \Rightarrow REGULI_SIMPTOME(RG, S)$$

Cheia acestei tabele este perechea (RG, S) , adică aceeași regulă nu poate fi legată de același simptom de două ori. În același timp, regula rămâne în `REGULI_DIAGNOSTIC`, iar simptomul rămâne în `SIMPTOME_CATALOG`. Astfel nu se repetă nici datele regulii, nici datele simptomului.

Concluzia este că schema finală este în BCNF pentru dependențele funcționale de mai sus, deoarece în fiecare tabel atributele depind de cheia tabelului. Pentru dependențele

multivaluate importante, schema respectă 4NF, deoarece valorile multiple au fost mutate în relații separate: abonamentele pacientului, afecțiunile pacientului și legătura dintre reguli și simptome.

4 Diagrama E/A sub formă UML

Diagrama următoare ilustrează entitățile extrase din etapa de normalizare, alături de constrângerile de chei primare (PK) și chei externe (FK).

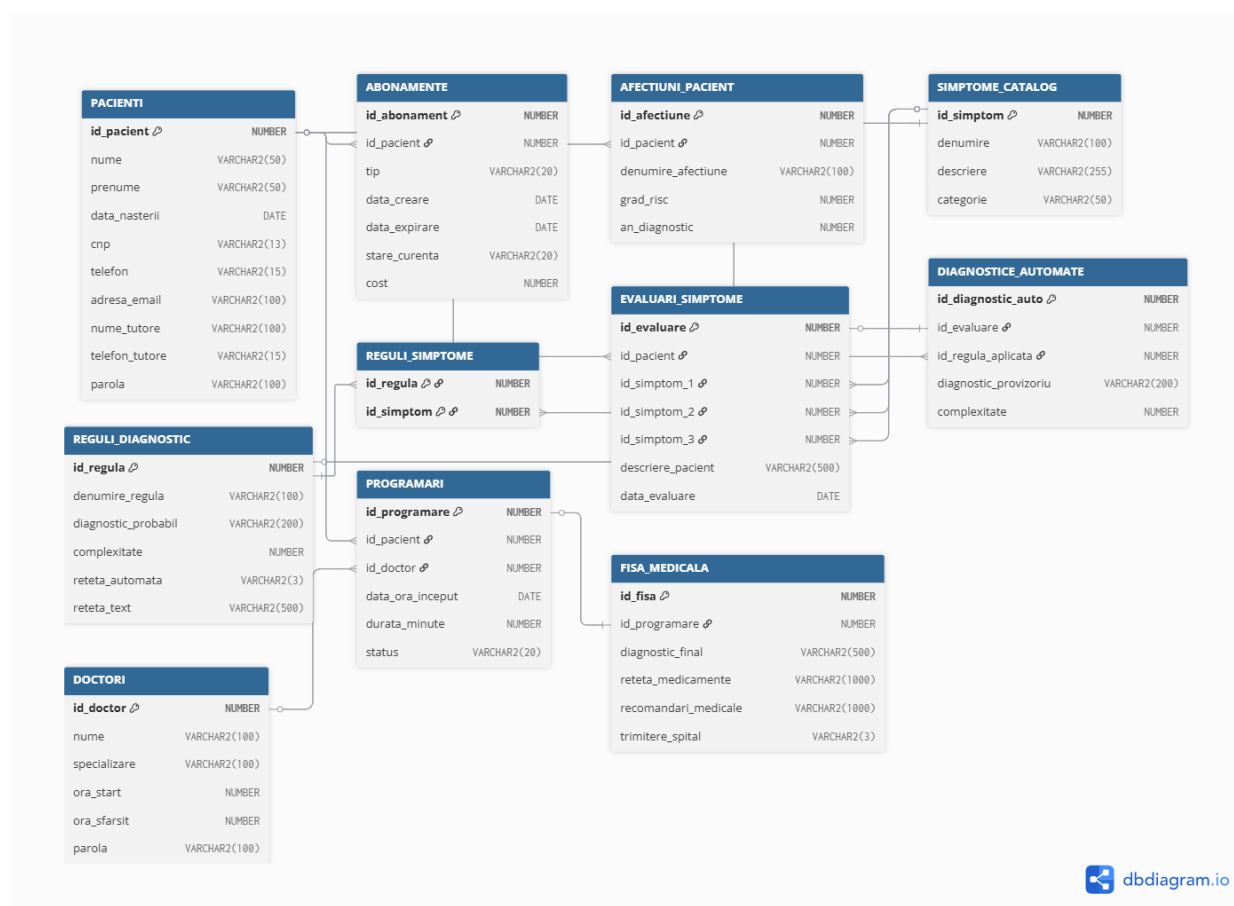


Figura 1: Diagrama UML a bazei de date

5 Entități, asocieri și chei

Entitățile principale rezultate în urma normalizării sunt PACIENTI, DOCTORI, ABONAMENTE, SIMPTOME_CATALOG, REGULI_DIAGNOSTIC, EVALUARI_SIMPTOME, DIAGNOSTICE_AUTOMATE, PROGRAMARI și FISA_MEDICALA.

Explicarea asocierilor (Relații 1:N, M:N, 1:1)

În implementarea bazei de date am folosit toate cele trei tipuri principale de cardinalități, traduse prin chei străine (Foreign Keys) pentru a respecta integritatea datelor:

- **Relații One-to-Many (1:N):** Aceasta este cea mai frecventă relație. Am folosit-o, de exemplu, între **PACIENTI** și **PROGRAMARI**. Un pacient unic poate avea, de-a lungul timpului, mai multe programări (1 la N), însă o programare individuală aparține strict unui singur pacient. Din punct de vedere tehnic, am plasat cheia externă **id_pacient** în tabela **PROGRAMARI**. Aceeași logică a fost aplicată pentru legătura dintre **DOCTORI** și **PROGRAMARI**, sau **PACIENTI** și **ABONAMENTE**.
- **Relații Many-to-Many (M:N):** Un simptom din catalog poate indica mai multe boli (reguli de diagnostic), iar o boală se manifestă de obicei prin mai multe simptome. Într-o bază de date relațională, o relație $M : N$ nu se poate implementa direct fără a încălca forma normală 1. De aceea, am "spart" această legătură folosind o tabelă de asociație (tabelă de joncțiune) numită **REGULI_SIMPTOME**. Aceasta deține o cheie primară compusă din două chei străine: (**id_regula**, **id_simptom**), asigurând astfel că o asociere regulă-simptom nu se dublează niciodată.
- **Relații One-to-One (1:1):** Această relație a fost utilizată pentru componentele care extind direct o altă tranzacție, dar au fost separate pentru a evita coloanele nule. De exemplu, între **PROGRAMARI** și **FISA_MEDICALA**. O programare finalizată generează o singură fișă medicală, iar o fișă medicală corespunde strict unei singure programări. Tehnic, am implementat acest lucru adăugând cheia externă **id_programare** în tabela **FISA_MEDICALA** și aplicând imediat o constrângere **UNIQUE** asupra ei. Astfel, este imposibil să se asigneze aceeași programare la două fișe medicale distincte. Același principiu se aplică între **EVALUARI_SIMPTOME** și **DIAGNOSTICE_AUTOMATE**.

Am folosit câmpuri de tip ID autoincrementat logic ca Primary Key (cheie surogat) deoarece sunt stabile și nu depind de informații care se pot modifica în lumea reală. CNP-ul este unic, dar nu l-am ales drept cheie primară, fiindcă este o dată personală sensibilă, iar în caz de eroare umană la înregistrare, modificarea unei chei primare propagate în multiple tabele ar genera un efort imens de actualizare în cascadă. De aceea, a rămas pur și simplu un câmp cu constrângerea de **UNIQUE**.

6 Constrângeri folosite

În proiect apar cele cinci tipuri de constrângeri discutate la baze de date:

- **constrângeri de domeniu:** de exemplu tip din **ABONAMENTE** poate fi doar **LUNAR** sau **ANUAL**;
- **constrângeri de cheie:** fiecare tabel important are primary key;
- **integritate a entității:** primary key-urile nu pot fi nule;
- **integritate referențială:** foreign key-urile nu permit legături către înregistrări inexistente;
- **constrângeri semantice:** trigger-ele verifică reguli precum programări valide, abonamente valide și tutore obligatoriu pentru minori.

7 Scriptul de creare a tabelelor

Mai jos este scriptul principal de creare a tabelelor. Am inclus structura tabelelor importante, pentru a se vedea cheile primare, cheile străine și constrângerile de bază.

```
1 CREATE TABLE PACIENTI (  
2     id_pacient NUMBER PRIMARY KEY,  
3     nume VARCHAR2(50) NOT NULL,  
4     prenume VARCHAR2(50) NOT NULL,  
5     data_nasterii DATE NOT NULL,  
6     cnp VARCHAR2(13) UNIQUE NOT NULL,  
7     telefon VARCHAR2(15) UNIQUE NOT NULL,  
8     adresa_email VARCHAR2(100) UNIQUE,  
9     nume_tutore VARCHAR2(100),  
10    telefon_tutore VARCHAR2(15),  
11    parola VARCHAR2(100) DEFAULT '1234' NOT NULL  
12 );  
13  
14 CREATE TABLE ABONAMENTE (  
15     id_abonament NUMBER PRIMARY KEY,  
16     id_pacient NUMBER NOT NULL REFERENCES PACIENTI(id_pacient),  
17     tip VARCHAR2(20) CHECK (tip IN ('LUNAR', 'ANUAL')),  
18     data_creatoare DATE DEFAULT SYSDATE NOT NULL,  
19     data_expirare DATE NOT NULL,  
20     stare_curenta VARCHAR2(20) DEFAULT 'ACTIV'  
21     CHECK (stare_curenta IN ('ACTIV', 'EXPIRAT', 'ANULAT')),  
22     cost NUMBER(10, 2) NOT NULL  
23 );  
24  
25 CREATE TABLE AFECTIUNI_PACIENT (  
26     id_afectiune NUMBER PRIMARY KEY,  
27     id_pacient NUMBER NOT NULL REFERENCES PACIENTI(id_pacient),  
28     denumire_afectiune VARCHAR2(100) NOT NULL,  
29     grad_risc NUMBER(1) CHECK (grad_risc BETWEEN 1 AND 5),  
30     an_diagnostic NUMBER(4)  
31 );  
32  
33 CREATE TABLE SIMPTOME_CATALOG (  
34     id_simptom NUMBER PRIMARY KEY,  
35     denumire VARCHAR2(100) NOT NULL,  
36     descriere VARCHAR2(255),  
37     categorie VARCHAR2(50)  
38     CHECK (categorie IN ('CARDIAC', 'RESPIRATOR', 'DIGESTIV',  
39     'NEUROLOGIC', 'PEDIATRIC', 'GENERAL'))  
40 );  
41  
42 CREATE TABLE REGULI_DIAGNOSTIC (  
43     id_regula NUMBER PRIMARY KEY,  
44     denumire_regula VARCHAR2(100) NOT NULL,  
45     diagnostic_probabil VARCHAR2(200) NOT NULL,  
46     complexitate NUMBER(1) CHECK (complexitate BETWEEN 1 AND 5),  
47     reteta_automata VARCHAR2(3) DEFAULT 'NU'  
48     CHECK (reteta_automata IN ('DA', 'NU')),  
49     reteta_text VARCHAR2(500)  
50 );  
51  
52 CREATE TABLE REGULI_SIMPTOME (  
53     id_regula NUMBER NOT NULL REFERENCES REGULI_DIAGNOSTIC(id_regula),
```

```

54     id_simptom NUMBER NOT NULL REFERENCES SIMPTOME_CATALOG(id_simptom),
55     PRIMARY KEY (id_regula, id_simptom)
56 );
57
58 CREATE TABLE EVALUARI_SIMPTOME (
59     id_evaluare NUMBER PRIMARY KEY,
60     id_pacient NUMBER NOT NULL REFERENCES PACIENTI(id_pacient),
61     id_simptom_1 NUMBER NOT NULL REFERENCES SIMPTOME_CATALOG(id_simptom),
62     id_simptom_2 NUMBER REFERENCES SIMPTOME_CATALOG(id_simptom),
63     id_simptom_3 NUMBER REFERENCES SIMPTOME_CATALOG(id_simptom),
64     descriere_pacient VARCHAR2(500),
65     data_evaluare DATE DEFAULT SYSDATE NOT NULL
66 );
67
68 CREATE TABLE DIAGNOSTICE_AUTOMATE (
69     id_diagnostic_auto NUMBER PRIMARY KEY,
70     id_evaluare NUMBER UNIQUE NOT NULL REFERENCES EVALUARI_SIMPTOME(id_evaluare),
71     id_regula_aplicata NUMBER NOT NULL REFERENCES REGULI_DIAGNOSTIC(id_regula),
72     diagnostic_provizoriu VARCHAR2(200) NOT NULL,
73     complexitate NUMBER(1) NOT NULL
74 );
75
76 CREATE TABLE DOCTORI (
77     id_doctor NUMBER PRIMARY KEY,
78     nume VARCHAR2(100) NOT NULL,
79     specializare VARCHAR2(100) NOT NULL,
80     ora_start NUMBER(2) CHECK (ora_start BETWEEN 0 AND 23),
81     ora_sfarsit NUMBER(2) CHECK (ora_sfarsit BETWEEN 0 AND 24),
82     parola VARCHAR2(100) DEFAULT '1234' NOT NULL
83 );
84
85 CREATE TABLE PROGRAMARI (
86     id_programare NUMBER PRIMARY KEY,
87     id_pacient NUMBER NOT NULL REFERENCES PACIENTI(id_pacient),
88     id_doctor NUMBER NOT NULL REFERENCES DOCTORI(id_doctor),
89     data_ora_inceput DATE NOT NULL,
90     durata_minute NUMBER(3) DEFAULT 20 NOT NULL,
91     status VARCHAR2(20) DEFAULT 'CONFIRMAT'
92     CHECK (status IN ('CONFIRMAT', 'ANULAT', 'FINALIZAT', 'FINALIZAT_AUTOMAT'))
93 );
94
95 CREATE TABLE FISA_MEDICALA (
96     id_fisa NUMBER PRIMARY KEY,
97     id_programare NUMBER UNIQUE NOT NULL REFERENCES PROGRAMARI(id_programare),
98     diagnostic_final VARCHAR2(500) NOT NULL,
99     reteta_medicamente VARCHAR2(1000),
100    recomandari_medicale VARCHAR2(1000),
101    trimitere_spital VARCHAR2(3) DEFAULT 'NU'
102    CHECK (trimitere_spital IN ('DA', 'NU'))
103 );

```

8 Logică PL/SQL implementată

Partea mai interesantă nu este un simplu IF pentru alegerea specializării, ci bucata în care baza de date caută automat o programare. Procedura `proceseaza_diagnostic` stabilește

mai întâi complexitatea cazului. Dacă nu este un caz simplu cu rețetă automată, atunci caută un doctor cu specializarea aleasă și verifică intervalele libere din următoarele zile.

Fragmentul de mai jos este relevant deoarece face o decizie de business la nivel de server de baze de date: nu doar inserează date, ci caută un slot valid ținând cont de doctor, programul lui și programările deja existente.

```
1 FOR v_zi IN 1..7 LOOP
2   FOR v_doc IN c_doctori(v_specializare_aleasa) LOOP
3     FOR v_ora IN v_doc.ora_start..(v_doc.ora_sfarsit - 1) LOOP
4       v_data_test := TRUNC(SYSDATE) + v_zi + (v_ora / 24);
5
6       SELECT COUNT(*) INTO v_gasit
7       FROM PROGRAMARI
8       WHERE id_doctor = v_doc.id_doctor
9             AND status = 'CONFIRMAT'
10            AND data_ora_inceput = v_data_test;
11
12       IF v_gasit = 0 THEN
13         v_id_doctor_ales := v_doc.id_doctor;
14         v_data_prog := v_data_test;
15         EXIT;
16       END IF;
17     END LOOP;
18
19     EXIT WHEN v_id_doctor_ales IS NOT NULL;
20   END LOOP;
21
22   EXIT WHEN v_id_doctor_ales IS NOT NULL;
23 END LOOP;
24
25 IF v_id_doctor_ales IS NULL THEN
26   RAISE exc_lipsa_locuri;
27 END IF;
```

Pentru utilizator, rezultatul este că nu trebuie să aleagă manual doctorul și ora. Aplicația caută prima variantă disponibilă, iar dacă nu găsește loc, ridică o excepție controlată. Mi se pare o parte mai potrivită de explicat în referat, deoarece folosește cursor, bucle imbricate, verificare de disponibilitate și inserare condiționată.

9 Prezentarea aplicației

9.1 Ecranul de login

Sistem Telemedicină

Spital General

Introduceți ID-ul (ex: 1)

Intră ca Pacient Intră ca Doctor

Nu ai cont? Creează unul nou

Creare Cont Pacient Nou

CNP (13 cifre)

Nume

Prenume

Data Nașterii

Telefon

Nume Tutor (doar pt minori)

Telefon Tutor (doar pt minori)

Alege Tipul de Abonament:

Selectează...

Inregistrare și Plată

Figura 2: Ecranul de autentificare pacient/doctor

În ecranul de login, utilizatorul poate intra ca pacient sau ca doctor. Pentru testare, autentificarea se face pe baza id-ului, deoarece proiectul este orientat mai mult pe partea de baze de date decât pe securitate.

9.2 Dashboard pacient

The screenshot shows a web application window titled "Dashboard Pacient - Oprea Vasile". The main content area displays a greeting "Bun venit, Oprea Vasile!" and a subscription status "Abonament valabil până la: 24.09.2026". Below this is a section titled "Istoricul tău medical:" containing a table with five columns: "Data", "Diagnostic Provizoriu", "Diagnostic Final", "Rețetă", and "Recomandări". The table has two rows of data and several empty rows below. The first row shows the date "22.05.2026 08:00 (Popescu)" and the diagnosis "[RISC CRONIC] Pneumonie virală". The second row shows the same date and the diagnosis "Aritmie minora - Necesita confirmare clinică". At the bottom of the dashboard, there are five buttons: "Evaluare Nouă", "Reînnoiește Abonament", "Refresh Date", "Log Out", and "Vezi Programarea Activă".

Data	Diagnostic Provizoriu	Diagnostic Final	Rețetă	Recomandări
22.05.2026 08:00 (Popescu)	[RISC CRONIC] Pneumonie virală			
22.05.2026 08:00 (Popescu)	Aritmie minora - Necesita confirmare clinică			

Figura 3: Dashboard pacient

Pacientul vede istoricul medical, diagnosticile provizorii, diagnosticile finale, rețetele și recomandările. Tot aici poate vedea valabilitatea abonamentului și poate porni o evaluare nouă.

9.3 Evaluarea de simptome

The screenshot shows a web application window titled "Evaluare Nouă". The main content area is titled "Evaluare Simptome" with a subtitle "Selectați simptomele și descrieți cum vă simțiți:". Below this, there are three sections for symptom selection: "Simptom 1 (Obligatori):", "Simptom 2 (Opțional):", and "Simptom 3 (Opțional):". Each section has a dropdown menu with the placeholder text "Alegeți simptomul...". Below these sections is a text area for "Descrieți în detaliu ce simțiți (Opțional):" with the placeholder text "Scrieți aici detalii despre starea dumneavoastră...". At the bottom of the form, there are two buttons: "Trimite Evaluarea" and "Anulare".

Figura 4: Formular evaluare simptome

În formularul de evaluare, primul simptom este obligatoriu, iar următoarele două sunt opționale. După trimitere, aplicația apelează pachetul PL/SQL care generează diagnosticul automat și, dacă este cazul, creează programarea.

9.4 Dashboard doctor

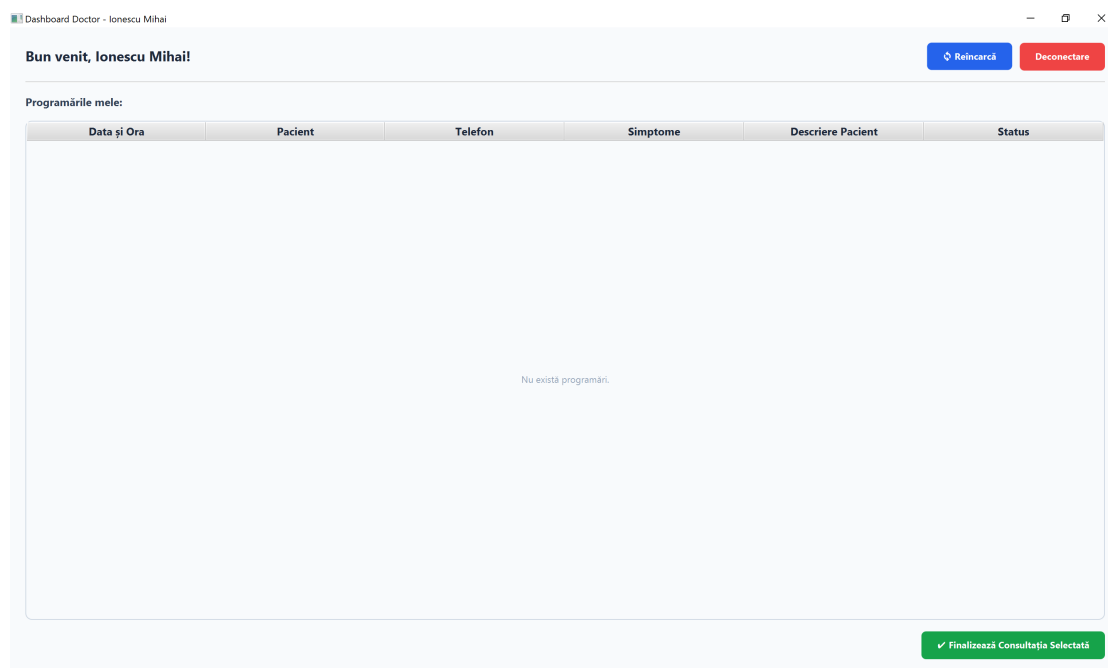


Figura 5: Dashboard doctor

Doctorul vede programările confirmate, simptomele pacientului și descrierea trimisă de pacient. După consultație, doctorul completează diagnosticul final, rețeta și recomandările. Dacă este nevoie, poate marca și trimiterea către spital.

10 Concluzie

Schema bazei de date a fost pornită de la o relație universală și apoi descompusă pe baza dependențelor funcționale și multivaluate. În forma finală, datele sunt separate pe entități clare, iar legăturile sunt făcute prin chei străine. Prin această descompunere se evită repetarea inutilă a datelor și se reduc anomaliiile de inserare, ștergere și modificare.

Pe partea de implementare, aplicația folosește PL/SQL pentru mai mult decât operații simple. Baza de date verifică reguli de business, generează diagnostice provizorii și caută automat programări valide. Din acest motiv, proiectul combină partea de modelare relațională cu partea de logică procedurală din Oracle.